

Practical work Machine Learning

TP #1 – Regression

**Professor
Abdelouahab Moussaoui**

Part 1 — Regression in action

Simple, Multiple and Polynomial linear regression models with R and Python.

Exercise 1 — Application of Simple Regression

In statistics, **linear regression** is a linear approach to modeling the relationship between a scalar response (or dependent variable) and one or more explanatory variables (or independent variables). In our example, we will use simple linear regression.

Simple linear regression is of the form $y = w \cdot x + b$, where y is the dependent variable, x is the independent variable, w and b are the training parameters that need to be optimized during the training process to obtain accurate predictions.

Now let's apply machine learning to train a dataset to predict **salary** from **years of experience**.

Application with R

Run the following script in R.

```
# Simple Linear Regression

# Importing the dataset
dataset = read.csv ( ' Salary_Data.csv ' )

# Splitting the dataset into the Training set and Test set
# install.packages('caTools')

library( caTools )

# Split dataset into training dataset (ratio 2/3) and testing dataset (ratio 1/3)
set.seed ( 123 )
split = sample.split( dataset $ Salary , SplitRatio = 2/3 )
training_set = subset ( dataset , split == TRUE)
test_set = subset ( dataset , split == FALSE)

# Feature Scaling
# training_set = scale(training_set)
# test_set = scale(test_set)

# Fitting Simple Linear Regression to the Training set
regressor = lm (formula = Salary ~ YearsExperience , data = training_set )

# Predicting the Test set results
y_pred = predict ( regressor , newdata = test_set )

# Visualizing the Training set results
library( ggplot2 )

ggplot () +
  geom_point (aes(x = training_set $ YearsExperience , y = training_set $ Salary ) ,
    color = ' red ' ) +
  geom_line (aes(x = training_set $ YearsExperience , y = predict( regressor , newdata =
    training_set )), color = ' blue ' ) +
  ggtitle ( ' Salary vs Experience (Training set) ' ) +
  xlab ( ' Years of experience ' ) +
  ylab ( ' Salary ' )

# Visualizing the Test set results
```

```
library( ggplot2 )
ggplot () +
  geom_point (aes(x = test_set $ YearsExperience , y = test_set $ Salary ),
    color = 'green') +
  geom_line (aes(x = training_set $ YearsExperience , y = predict ( regressor , newdata
= training_set )), color = ' blue ' ) +
  ggtitle ( 'Salary vs Experience (Test set) ' ) +
  xlab ( ' Years of experience ' ) +
  ylab ( ' Salary ' )
```

Application with Python

Run the following script in anaconda python.

Step 1: Import Libraries

In this first step, we will import the **pandas library** which will be used to store the data in a Pandas DataFrame. The **matplotlib** is used to plot graphs.

```
import numpy as np
import matplotlib.pyplot as plt
from pandas as pd
```

Step 2: Import the dataset

In this step, we will load the dataset that contains the data under the name " **Salary_Data.csv** ". The variable **X** will store the " **years of experience** " and the variable **Y** will store the " **salary** ". The **dataset.head(5)** is used to visualize the first 5 rows of data.

```
dataset = p.d. read_csv ( ' Salary_Data.csv ' )
X = dataset . iloc [ : , : -1]. values
y = dataset . iloc [ : , -1]. valuesdataset . head( 5 )
```

```
>>
YearsExperience Salary
1.1 39343.0
1.3 46205.0
1.5 37731.0
2.0 43525.0
2.2 39891.0
```

Step 3: Split the dataset into training set and test set

In this step, we need to divide the dataset into the training set, on which the linear regression model will be trained and the test set, on which the trained model will be applied to visualize the results. In this, the **test_size=0.2** indicates that **20%** of the data will be kept as **the test set** and the remaining **80%** will be used for training as **the learning set** .

```
from sklearn . model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split (X, y, test_size = 0.2 )
```

Step 4: Train the simple linear regression model on the training set

LinearRegression class is imported and assigned to the variable "**regressor**". The **regressor.fit()** function is equipped with **X_train** and **Y_train** on which the model will be trained.

```
from sklearn . linear_model import LinearRegression
regressor = LinearRegression ()
regressor.fit ( X_train , y_train )
```

Step 5: Predict the results of the test set

In this step, the `regressor.predict()` function is used to predict the values of the test set and the values are stored in the variable `y_pred`.

```
y_pred = regressor. predict ( X_test )
```

Step 6: Comparing the test set with the predicted values

In this step, a Pandas DataFrame is created to compare the salary values of the original test set (`y_test`) and the predicted outcomes (`y_pred`).

```
df = p.d. DataFrame ({ 'Real Values' : y_test, 'Predicted Values' : y_pred})
df
>>
Real Values Predicted Values
109431.0 107621.917107
81363.0 81508.217112
93940.0 82440.849255
55794.0 63788.206401
66029.0 74047.159970
91738.0 89901.906396
```

We can see that the predicted salaries are very close to the actual salary values and we can conclude that the model has been well trained.

Step 7: View the results

actual and **predicted** salary values as well as the linear regression line on a plotted graph.

```
plt. scatter (X_test, y_test, color = ' red ' )
plt. scatter (X_test, y_pred, color = ' green ' )
plt. plot (X_train, regressor. predict (X_train), color = ' black ' )
plt. title ( ' Salary vs Experience (Result) ' )
plt. xlabel ( ' YearsExperience ' )
plt. ylabel ( ' Salary ' )
plt. show ()
```



Salary vs. experience

In this graph, the actual values are plotted in " **Red** " color and the predicted values are plotted in " **Green** " color. The linear regression line that is generated is drawn in " **Black** " color.

Exercise 2 — Application of Multiple Regression

In this data, we have the four independent variables, namely *R&D Expenses*, *Administration*, *Marketing Expenses*, and *State*. There is one independent variable, i.e., *Profit*. So our job is to train the ML model with this data to understand the correlation between each of the four features (or independent variables) and predict a profit for another new company with all this data.

Application with R

Run the following script in R.

```
# Multiple Linear Regression

# Importing the dataset
dataset = read.csv ( ' 50_Startups.csv ' )

# Encoding categorical data
dataset $ State = factor(dataset$State,
levels = c('New York', 'California', 'Florida'),
labels = c(1, 2, 3))

# Splitting the dataset into the Training set and Test set
# install.packages('caTools')

library( caTools )
set.seed ( 123 )

# Split dataset into training dataset (ratio 80%) and testing dataset (ratio 20%)
split = sample.split( dataset $ Profit , SplitRatio = 0.8 )
training_set = subset ( dataset , split == TRUE)
test_set = subset ( dataset , split == FALSE)

# Feature Scaling
# training_set = scale(training_set)
# test_set = scale(test_set)

# Fitting Multiple Linear Regression to the Training set
regressor = lm (formula = Profit ~ . , data = training_set )

# Predicting the Test set results
y_pred = predict ( regressor , newdata = test_set )
```

Application with Python

Run the following script in anaconda python.

Step 1: Import Libraries

In this first step, we will import the libraries needed to create the ML model . The libraries **Numpy** and **matplotlib** are imported . In addition, we imported the **pandas** library for data analysis.

```
import numpy as np
import matplotlib.pyplot as plt
from pandas as pd
```

Step 2: Import the dataset

In this next one we will use pandas to store the data load which we will load from the file " **50_Startups.csv** " in the form of a **DataFrame** from the **pandas** library named " **dataset** " using the function " **pd.read_csv** " .

We loop through our dataset and assign the independent variable (x) to the first four columns of our dataset, namely **R&D Spend** (**index=0**), **Administration** (**index=1**), **Marketing Spend** (**index=2**) and **State** (**index=3**).

```
dataset = p.d. read_csv ( ' 50_Startups.csv ' )
X = dataset . iloc [ : , : -1]. values
y = dataset . iloc [ : , -1]. valuesdataset . head( 5 )

>>
R&D Spend Administration Marketing Spend State Profit165349.20 136897.80 471784.10
New York 192261.83162597.70 151377.59 443898.53 California 191792.06153441.51
101145.55 407934.54 Florida 191050.39144372.41 118671.85 383199.62 New York
182901.99142107.34 91391.77 366168.42 Florida 166187.94
```

We use the corresponding **.iloc** function to slice the **DataFrame** to assign these indexes to **X** . Here we use **[:, :-1]** which can be interpreted as **[include all rows, include all columns up to -1 (excluding -1)]** . In this, **-1** refers to the first column from the last. So, we assign the 0th, 1st, 2nd and 3rd columns as **X** .

We assign the last column (**-1**) to the dependent variable which is **y** . We print the **DataFrame** to see if we have the correct columns for our training data.

Step 3: Encoding categorical data

As long as there are numbers in the dataset, we can easily apply mathematical calculations on the dataset and create prediction models. In this dataset, we encounter a non-numeric variable which is " **State** " This is also called **categorical data** .

We encode this categorical data using another important library called **sklearn** . In this, we import the **ColumnTransformer** and **OneHotEncoder** . The **ColumnTransformer** allows a particular column of the **DataFrame** to be transformed separately. In our case, we use the **OneHotEncoder** to transform our " **State** " column (**index=3**) into numeric data.

After encoding the categorical data, we print our **DataFrame X** and see the changes. We see that three new columns have been added at the beginning. Each column represents one of the " **states** ." For example, in the first row, the third column represents " **New York** ," and therefore the value "1" in the third column.

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
```

```
ct = ColumnTransformer (transformers = [ ( ' encoder ' , OneHotEncoder() , [ 3 ] ) ],
                             remainder = ' passthrough ' )
np . array ( ct.fit_transform ( X ) )
```

Step 4: Split the dataset into training set and test set

Once our dataset is ready, the next important task is to divide our dataset into a training set and a test set. We do this in order to train our model with a part of the data called the "**training set**" and test the prediction results on another dataset called the "**test set**".

`train_test_split` function to split our data. Here we give the "`test_size = 0.2`", which indicates that **20%** of the data is the **test set**. In our case, **10** random seed data will be chosen as the test set and remaining **40** seed data will be chosen for the **training set**.

```
from sklearn . model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split (X, y, test_size = 0.2 )
```

Step 5: Train the multiple linear regression model on the training set

In the next step, we import the "**LinearRegression**" class that will be applied to our training set. We assign a variable "**regressor**" to the **LinearRegression** class. We then use "**regressor.fit**" to fit the training dataset (`X_train` and `y_train`) to this **LinearRegression** class for the training process to occur.

```
from sklearn . linear_model import LinearRegression
regressor = LinearRegression ()
regressor.fit ( X_train , y_train )
```

Step 6: Predict the test set results

In the next step, we will predict the profit of the test set using the trained model, namely "regressor". The actual values (profits) of the test set data (`X_test`) are stored in the variable `y_test`.

regressor.predict function to predict the values of our test data `X_test`. We assign the predicted values as `y_pred`. We now have two data points, `y_test` (actual values) and `y_pred` (predicted values).

```
y_pred = regressor. predict ( X_test )
```

Step 7: Comparing the test set with the predicted values

In this step, we will print both the `y_test` values as **actual values** and the `y_pred` values as **predicted values** of each `X_test` in a pandas DataFrame. This way, we get the values for all **10** `X_test` data.

```
df = p.d. DataFrame ({ 'Real Values' : y_test, 'Predicted Values' : y_pred})
df
>>
Real Values Predicted
Values 78239.91
74963.602167 182901.99 1 73144.548525 64926.08 45804.248438 105733.54 1 08530.843936
141585.52 1 27674.466487 108552.04 1 11471.421444 146121.95 1 33618.038644 105008.31
1 14655.651664 96778.92 96466.443219 97483.56
```

In the first row, the actual values have a value of **78239.91** and the predicted values have a value of **74963.60** . We see that the model has closely predicted this value and hence we can say that our model has good accuracy.

Exercise 3 — Application of Polynomial Regression

In this data, we have the two independent variables namely, *Position* and *Level* . There is one independent variable, i.e., *salary* . So, in this problem, we need to train a **polynomial** regression model with this data to understand the correlation between the level and salary of employee data in the company and be able to predict the salary (output variable **Salary**) of the new employee based on this data.

Application with R

Run the following script in R.

```
# Polynomial Regression

# Importing the dataset
dataset = read.csv ( ' Position_Salaries.csv ' )
dataset = dataset [ 2:3 ]

# Splitting the dataset into the Training set and Test set
# # install.packages('caTools')
# library(caTools)
# set.seed(123)
# split = sample.split(dataset$Salary, SplitRatio = 2/3)
# training_set = subset(dataset, split == TRUE)
# test_set = subset(dataset, split == FALSE)

# Feature Scaling
# training_set = scale(training_set)
# test_set = scale(test_set)

# Fitting Linear Regression to the dataset
lin_reg = lm (formula = Salary ~ . , data = dataset )

# Fitting Polynomial Regression to the dataset
dataset $ Level2 = dataset $ Level ^2
dataset $ Level3 = dataset $ Level ^3
dataset $ Level4 = dataset $ Level ^4
poly_reg = lm (formula = Salary ~ . ,
data = dataset )

# Visualizing the Linear Regression results
# install.packages('ggplot2')
library( ggplot2 )
ggplot ( ) +
  geom_point (aes(x = dataset $ Level , y = dataset $ Salary ),
color = 'red') +
  geom_line (aes(x = dataset $ Level , y = predict ( lin_reg , newdata = dataset )),
color = 'blue') +
  ggtitle ( ' Truth or Bluff (Linear Regression) ' ) +
  xlab ( ' Level ' ) +
  ylab ( ' Salary ' )

# Visualizing the Polynomial Regression results
# install.packages('ggplot2')
```

```

library( ggplot2 )
ggplot () +
  geom_point (aes(x = dataset $ Level , y = dataset $ Salary ),
color = 'red') +
  geom_line (aes(x = dataset $ Level , y = predict ( poly_reg , newdata = dataset )),
color = 'blue') +
  ggtitle ( 'Truth gold Bluff (Polynomial Regression) ' ) +
  xlab ( ' Level ' ) +
  ylab ( ' Salary ' )

# Visualizing the Regression Model results (for higher resolution and smoother curve)
# install.packages('ggplot2')
library( ggplot2 )
x_grid = seq( min ( dataset $ Level ), max ( dataset $ Level ), 0.1 )
ggplot () +
  geom_point (aes(x = dataset $ Level , y = dataset $ Salary ),
color = 'red') +
  geom_line (aes(x = x_grid , y = predict ( poly_reg ,
newdata = data.frame( Level = x_grid ,
Level 2 = x_grid ^2 ,
Level 3 = x_grid ^3 ,
Level 4 = x_grid ^4 ))),
color = 'blue') +
  ggtitle ( ' Truth or Bluff (Polynomial Regression) ' ) +
  xlab ( ' Level ' ) +
  ylab ( ' Salary ' )

# Predicting a new result with Linear Regression
predict ( lin_reg , data.frame( Level = 6.5 ))

# Predicting a new result with Polynomial Regression
predict ( poly_reg , data.frame( Level = 6.5 ,
Level 2 = 6.5^2 ,
Level 3 = 6.5^3 ,
Level 4 = 6.5^4 ))

```

Application with Python

Run the following script in anaconda python.

Step 1: Import Libraries

In this first step, we will import the libraries needed to create the ML model . The libraries *numpy* And *matplotlib* are imported . In addition, we must import the *pandas* library for data access and analysis.

```

import numpy as np
import matplotlib.pyplot as plt
from pandas as pd

```

Step 2: Import the dataset

In this step, we will use pandas to store the data obtained from my github repository and store it as a Pandas DataFrame using the " *pd.read_csv* " function.

We loop through our dataset and assign the independent variable (**x**) to the second column with the column name " **Level** " and the dependent variable (**y**) in the last column, which is the " **Salary** " to predict.

```
dataset = p.d. read_csv ( ' PositionSalaries_Data.csv ' )
X = dataset . iloc [ : , 1:-1]. values
y = dataset . iloc [ : , -1]. values
dataset . head( 5 )

>>
Position Level SalaryBusiness Analyst 1 45000Junior Consultant 2 50000Senior
Consultant 3 60000Manager 4 80000Country Manager 5 110000
```

We use the corresponding `.iloc` function to slice the **DataFrame** to assign these indexes to **X** and **Y** . In this, *Level* is taken as the independent variable and is assigned to **X** . The dependent variable to be predicted is the last column (**-1**) which is *Salary* and it is assigned to **y** . We print the **DataFrame** " **dataset** " to see if we have the right columns for our training data.

Step 3: Train the polynomial regression model on the dataset

The dataset we are using has very few lines and hence we train the dataset to build the polynomial regression model. In this, the " **PolynomialFeatures** " function is used to assign the degree of the polynomial line we are going to plot. In this case, the degree is set to **4** .

The independent variable **X** is then fitted with the **PolynomialFeatures** class and is converted into a new variable **X_poly** . In this, the variable **X** is converted into a new matrix **X_Poly** which consists of all polynomial combinations of features of **degree = 4** .

```
from sklearn . preprocessing import PolynomialFeatures
from sklearn . linear_model import LinearRegression
poly_reg = PolynomialFeatures (degree = 4 )
X_poly = poly_reg. fit_transform ( X )
lin_reg = LinearRegression ()
lin_reg.fit ( X_poly, y )
```

The class " **LinearRegression** " is also imported and is assigned to the variable " **lin_reg** " which is equipped with the **X_poly** and **y** for building the model.

Step 4: Predict the outcomes

In this step, we will predict the **Salary values** based on the constructed polynomial regression model. The function " **regressor.predict** " is used to predict the values of our independent variable, **X_poly** . We assign the predicted values as **y_pred** . We now have two data, **y** (actual values) and **y_pred** (predicted values).

```
y_pred = lin_reg. predict ( X_poly )
```

Step 5: Compare actual values with predicted values

In this step, we will print both the **y** values as **actual values** and the **y_pred** values as **predicted values** of each **X_test** in a **Pandas DataFrame**.

```
df = p.d. DataFrame ({ 'Real Values' : y, 'Predicted Values' : y_pred})
df
```

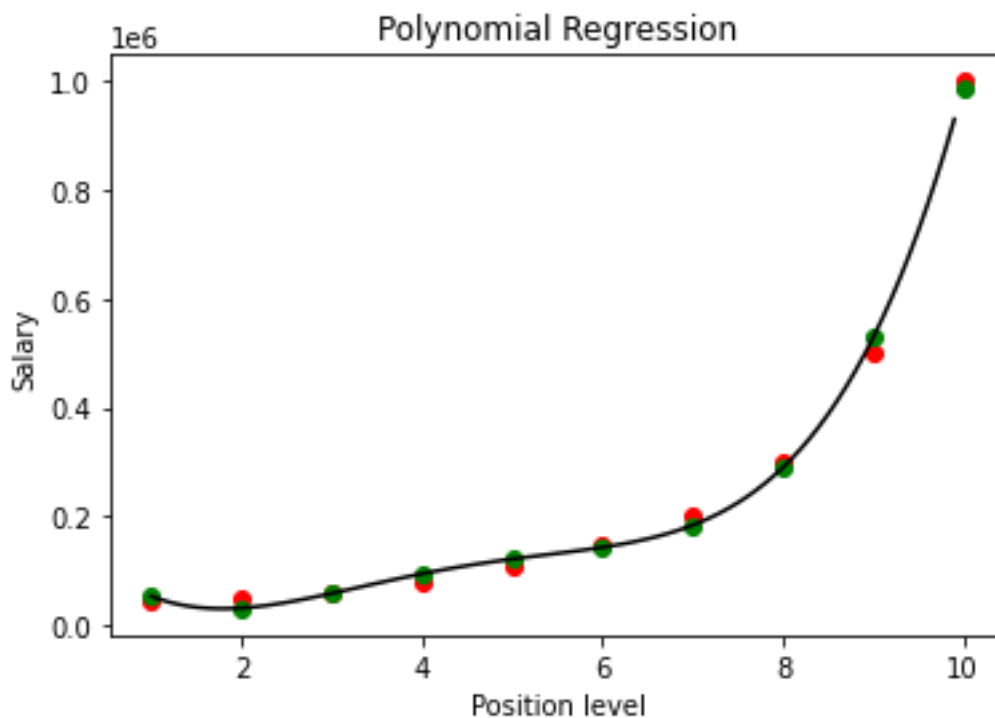
```
>>
Real Values      Predicted
Values 45000 53356.643357      50000
31759.906760      60000
58642.191142      80000
94632.867133      110000 1
21724.941725      150000 1
43275.058275      200000 1
84003.496504      300000 2
89994.172494      500000 5
28694.638695      1000000 9
```

We can see that the model did a great job fitting the data and predicting the employee's salary based on the job level.

Step 6: Visualizing the Polynomial Regression Results

In this last step, we will visualize the polynomial model that was built using the given data and plot the values of " **y** " and " **y_pred** " on the graph and analyze the results

```
X_grid = np.arange ( min ( X ) , max ( X ) , 0.1 )
X_grid = X_grid . reshape (( len ( X_grid ) , 1 ))
plt . scatter ( X, y, color = ' red ' )
plt . scatter ( X, y_pred, color = ' green ' )
plt . plot ( X_grid, lin_reg. predict (poly_reg. fit_transform (X_grid )) , color = '
black ' )
plt. title ( ' Polynomial Regression ' )
plt. xlabel ( ' PositionLevel ' )
plt.ylabel ( ' Salary ' )
plt. show ()
```



Polynomial regression model

In this graph, the actual values are plotted in color "**Red**" » and the predicted values are plotted in color "**Green**". The polynomial regression line that is generated is drawn in "**black**" color.

Part 2 — Application of Regression

Exercise 1 — Application of Simple Regression

DataHeight.txt file contains the age and height of **50** children, respectively. The objective of this work is to show that there is a linear relationship between the age and height of children. To achieve this, you are asked to perform the following steps:

Questions :

1. Load the data into a variable named **tab** .
2. Load the **age vector** in a variable named **x** .
3. **height** vector into a variable named **y** .
4. Give the distribution of y as a function of x (use the plot and **qplot functions**).
5. Looking at the graph, can we say that there is a correlation between the two variables? Justify.
6. Prove in a statistical (rigorous) way that x and y are either correlated or not?
7. What are the values of the parameters (θ_0 and θ_1).
8. What is the value of the residual error?
9. Give the equation of the regression line.
10. Draw the regression line.
11. According to the model found to predict the weights of 4 children aged 3, 7, 9 and 12 years respectively.
12. Predict the height of a 10-year-old child with a 95% confidence interval.

Exercise 2 — Multiple & Polynomial Regression

We want to study the respiratory problems of a population of **725** individuals in order to predict the causes of these problems. For this purpose, we have **6 variables** named respectively **LungCap** , **Age** , **Height** , **Smoke** , **Caesarean** representing respectively the respiratory capacity, age, height, whether the individual is a smoker or not and finally whether he had a normal birth or by cesarean section.

It is requested to run the following R program to interpret the results of each step.

```
LungCapData <- read.table (file.choose(), header = T, sep="\t")
attach (LungCapData)
names (LungCapData)
class (Age)
class (Smoke)
levels (Smoke)
modell1 <- lm ( LungCap ~ Age + Height )
summary (modell1)
horn (Age, Height, method="pearson")
confined (modell1, coef.level=0.95)
modell2 <- lm(LungCap ~ Age + Height + Smoke + Gender + Caesarean)
summary (modell2)
plot (modell2)
```

Exercise 3 — Polynomial Regression

This time, we hope to find a connection between one or more variables in the Boston database, which you'll find in the MASS package. This database tells us about the value of a home in the suburbs of Boston (USA). It contains records of variables such as:

- **crim**: percentage of criminal residents per city.
- **rad** : index accessibility to highways radials.
- **black** : proportion of blacks per 1000 individuals and by city.
- **lstat** : minimal status of the population (percentage).
- **medv** : median value of residents-owners in \$1000 per unit.
- Etc

Questions :

1. Show information.
2. Display the type of each data.
3. What can you say about this data?
4. Divide the data using the 400 first observations that the training data and the rest inasmuch as data of test.
5. Check if there is a relationship linear between **medv** floor.
6. Draw the scatter plot of these two variables.
7. Do the same with **medv** and **lstat**.
8. Draw the regression line.
9. Check statically and graphically whether the variable **medv** can be explained by a simple linear model using **lstat**.
10. What can you deduce?
11. Execute and comment out the `pairs(Boston)` instruction.
12. Repeat the instruction but only with the first, third and seventh variables.
13. use the training dataset to train the multiple linear model with **medv** as the explained variable and **lstat** and **age** as the explanatory variables.
14. What can we conclude? Explain
15. use the training dataset to train the multiple linear model with **medv** as the explained variable and the logarithm of the **lstat** variable and the **age** variable as explanatory variables.
16. What can we conclude? Explain
17. Check the linear relationship **medv** between and all other variables.
18. What can we conclude? Explain
19. Reconstruct the linear model with all variables except the non-significant ones.
20. What can we conclude? Explain
21. Build the linear model with the variable **medv** and the interaction between **lstat** and **age**.
22. Check if the model is nonlinear of degree 2, 3, 4, 5, 6 and 7.
23. What can we conclude?